

Βαθιά Μάθηση και Ανάλυση Πολυμεσικών Δεδομένων

Αναφορά Εργασίας

Γεννήτρια Εικόνων Λουλουδιών από Κείμενο:

Ένα Conditional GAN με Transformer Encoder, βελτιστοποιημένο μέσω CLIP-
Guided Reinforcement Learning

Συνδιασμός Convolutional, Transformer και Reinforcement Learning Αρχιτεκτονικών

Μεταπτυχιακό πρόγραμμα Τεχνητής Νοημοσύνης

Ονοματεπώνυμο: Αθανάσιος Εμμανουηλίδης

AEM: 211

Περίληψη

Αυτή η εργασία παρουσιάζει ένα ολοκληρωμένο pipeline για την δημιουργία κειμένου σε εικόνα για το σύνολο δεδομένων Oxford Flowers 102. Είναι σχεδιασμένο ώστε να ικανοποιεί την απαίτηση ανάθεσης εργασίας και με τα τρία βασικά παραδείγματα βαθιάς μάθησης: Συνελικτικά Δίκτυα, Μετασχηματιστές και Ενισχυτική Μάθηση. Αντί να αντιμετωπίζονται αυτά ως τρία ξεχωριστά μικρά έργα, ενσωματώθηκαν σε ένα ενιαίο συνεκτικό σύστημα.

Στη Φάση 1, υλοποιήθηκε ο transformer encoder, που υλοποιήθηκε από την αρχή ακολουθώντας εν μέρη τις τεχνικές και την δομή που περιεγράφηκε στην ακαδημαϊκή εργασία Attention Is All You Need. Το αρχικό σύνολο δεδομένων από μόνο του δεν παρείχε περιγραφές. Μια ομάδα που ασχολήθηκε πάνω στην παραγωγή περιγραφών παρείχε μια εμπλουτισμένη έκδοχή (πηγή). Στόχος του encoder είναι να κωδικοποιεί περιγραφές λουλουδιών φυσικής γλώσσας σε πυκνά διανύσματα (dense vectors) των 256 διαστάσεων. Αυτά τα διανύσματα θα περάσουν στην συνέχεια σε ένα Deep Convolutional GAN (Γεννήτρια και Διακριτής) το οποίο εκπαιδεύτηκε σε 6.070 ζεύγη εικόνας-περιγραφής/λεζάντας σε ανάλυση 64*64. Στον κωδικοποιητή(encoder) εφαρμόζεται το InfoNCE (Information Noise-Contrastive Estimation) έτσι ώστε να αποφευχθεί η καταστροφή της αναπαράστασης των εικόνων, ενώ ένα αρνητικό ζεύγος λανθασμένου κειμένου δίνεται στον Discriminator ώστε να μπορέσει να ευθυγραμμίσει κείμενο και εικόνα. Το μοντέλο Φάσης 1 παράγει αναγνωρίσιμα, πολύχρωμα λουλούδια με μερική προσαρμογή κειμένου.

Στη Φάση 2, η γεννήτρια της Φάσης 1 βελτιστοποιείται μέσω της καθοδήγησης από το μοντέλο CLIP (Contrastive Language-Image Pre-training). Η γεννήτρια (Generator) διαμορφώνεται ως μια ντετερμινιστική πολιτική και χρησιμοποιούμε την βαθμολογία ομοιότητας εικόνας-κειμένου του CLIP ως σήμα ανταμοιβής. Μέσω του Deterministic Policy Gradient (DPG) με τον Discriminator να λειτουργεί παγωμένος μετά την εκπαίδευση της πρώτης φάσης, γίνεται η ευθυγράμμιση του σημασιολογικού κειμένου και έτσι μπόρεσε να βελτιωθεί η βασική βαθμολογία CLIP 0,252 σε 0,520, μια σχετική βελτίωση +106%, διατηρώντας παράλληλα τον ρεαλισμό της εικόνας.

Αυτή η αναφορά καταγράφει τις αποφάσεις σχεδιασμού, τις αρχιτεκτονικές λεπτομέρειες, τον εντοπισμό των σφαλμάτων που απαιτήθηκε να επιλυθούν (προβλήματα με τον κωδικοποιητή, με τον θόρυβο αλλά και πολλαπλά ανεπιτυχή ενδιάμεσα πειράματα) και τα τελικά ποσοτικά και ποιοτικά αποτελέσματα.

Πίνακας Περιεχομένων

Περίληψη.....	2
1. Εισαγωγή και Πεδίο Εφαρμογής του Έργου.....	5
1.1 Στόχος.....	5
2. Προετοιμασία του συνόλου δεδομένων.....	6
2.1 Επιλογή του συνόλου δεδομένων.....	6
2.2 Προεπεξεργασία εικόνων.....	6
2.3 Ρυθμίσεις του Dataloader.....	6
3. Κατασκευή Tokenizer.....	7
3.1 Γιατί έφτιαξα τον δικό μου tokenizer.....	7
3.2 Λεπτομέρειες υλοποίησης.....	7
3.3 Επαλήθευση αποτελεσματικότητας του tokenizer.....	7
4. Transformer Encoder (από την αρχή).....	8
4.1 Στόχος.....	8
4.2 Αρχιτεκτονική.....	8
4.2.1 Token + Positional Embedding.....	8
4.2.2 Multi-Head Attention.....	8
4.2.3 Position-wise Feed-Forward Network.....	8
4.2.4 Encoder Block.....	8
4.3 Επαλήθευση.....	9
5. Conditional GAN (Φάση 1).....	10
5.1 Vanilla DCGAN (μόνο demo).....	10
5.2 Αρχιτεκτονική generator.....	10
5.3 Αρχιτεκτονική Discriminator.....	10
5.4 Συνάρτηση σφάλματος.....	11
5.5 Επιπλέον λεπτομέρειες εκπαίδευσης.....	11
6. Εκπαίδευση Σταδίου πρώτου: Διαδικασία, Αποτυχίες και Τελικά Αποτελέσματα.....	12
6.1 Τι λειτούργησε την πρώτη φορά.....	12
6.2 Σφάλματα που παρατηρήθηκαν κατά την εκπαίδευση.....	12
6.2.1 Πρώτη προσπάθεια: Κυριαρχία του D, κατάρρευση του G.....	12
6.2.2 Δεύτερη προσπάθεια: περίπλοκη αρχιτεκτονική.....	12
6.2.3 Τρίτη προσπάθεια: ένα σφάλμα που ήταν “κριμένο” στα data.....	12
6.2.4 Τέταρτη προσπάθεια: encoder collapse.....	12
6.2.5 Πέμπτη προσπάθεια: το warmup που ήταν τελικά επιβλαβές.....	12
6.2.6 Τελική διόρθωση.....	13
6.3 Τελική εκπαίδευση.....	13
6.4 Ποιότητα αποτελεσμάτων πρώτης φάσης.....	13
6.5 Περιορισμοί που εμφανίστηκαν στην πρώτη φάση.....	15
7. Εκπαίδευση Φάσης 2: Ενισχυτική Μάθηση (Reinforcement Learning).....	16

7.1 Γιατί χρειαζόμαστε RL σε αυτήν την εργασία.....	16
7.2 Διατύπωση του προβλήματος.....	16
7.3 Πειραματική Υλοποίηση.....	16
7.4 Γιατί πάγωσα τον Discriminator.....	17
8. Σύγκριση και αποτελέσματα της δευτερης φάσης.....	18
8.1 Εκπαίδευση.....	18
8.2 Ποσοτική σύγκριση.....	18
8.3 Ποιοτική σύγκριση.....	19
8.4 Συγκριτικά αποτελέσματα των δύο φάσεων (οπτικά).....	20
10. Τελικά συμπεράσματα.....	21
11. References.....	22

1. Εισαγωγή και Πεδίο Εφαρμογής του Έργου

1.1 Στόχος

Η εργασία απαιτούσε να λυθούν τρία προβλήματα βαθιάς μάθησης: Βαθιά Συνελικτικά Νευρωνικά Δίκτυα, RNN ή Transformers και Ενισχυτική Μάθηση (Reinforcement Learning). Οι οδηγίες επέτρεπαν τον συνδυασμό δύο ή τριών από αυτά τα παραδείγματα σε ένα ενιαίο μεγαλύτερο έργο, και αυτή είναι η διαδρομή που επέλεξα. Η εργασία αυτή έχει ως σκοπό την δημιουργία εικόνας λουλουδιού με βάση μια περιγραφή που θα δίνει ο χρήστης. Ένας χρήστης παρέχει μια περιγραφή σε φυσική γλώσσα (π.χ. "αυτό το λουλούδι έχει κόκκινο χρώμα, με πέταλα που είναι μπλέ") και το σύστημα παράγει μια εικόνα RGB 64*64 ενός λουλουδιού που ταιριάζει με αυτήν την περιγραφή. Αυτό το πρόβλημα απαιτεί φυσικά και τα τρία παραδείγματα:

- Ένας transformer encoder ο οποίος θα μετατρέπει την περιγραφή αυτή κειμένου σε μια διανυσματική αναπαράσταση.
- Ένα συνελικτικό GAN (Generator + Discriminator) όπου θα μετατρέπει το διάνυσμα + τυχαίο θόρυβο σε μια ρεαλιστική εικόνα λουλουδιού.
- Μια φάση βελτιστοποίησης της Ενισχυτικής Μάθησης που προσπαθεί να βελτιώσει περαιτέρω τις εξόδους του generator με το κείμενο χρησιμοποιώντας το CLIP ως μοντέλο που θα υπολογίζει την ανταμοιβή.

2. Προετοιμασία του συνόλου δεδομένων

2.1 Επιλογή του συνόλου δεδομένων

Χρησιμοποίησα το σύνολο δεδομένων Oxford Flowers 102, το οποίο περιέχει 8.189 εικόνες λουλουδιών σε 102 κατηγορίες. Το καθαρό σύνολο δεδομένων PyTorch torchvision Flowers102, ωστόσο, περιέχει μόνο ετικέτες κλάσεων και καθόλου περιγραφές σε φυσική γλώσσα, επομένως δεν μπορεί να χρησιμοποιηθεί απευθείας για την εργασία.

Λόγο της φύσης του προβλήματος ήταν απαραίτητο να βρεθούν λεκτικά για κάθε εικόνα. Για αυτόν τον λόγο έγινε χρήση του βοηθητικού συνόλου που δημοσιεύτηκε με τους Reed et al. 2016

"Generative Adversarial Text to Image Synthesis", το οποίο παρέχει ~10 λεζάντες ανά εικόνα λουλουδιού. Οι λεζάντες και οι εικόνες έπρεπε να ενωθούν χειροκίνητα και οι αρχικές λεζάντες αποθηκεύτηκαν σε δυαδική μορφή Torch-7 .t7 την οποία το PyTorch δεν μπορεί να διαβάσει απευθείας. Ως εκ τούτου, δημιούργησα μια προσαρμοσμένη κλάση FlowersTextDataset που:

1. Διαβάζει τη λίστα κλάσεων trainclasses.txt.
2. Περιηγείται στον φάκελο με τις περιγραφές και συλλέγει ζεύγη (image_path, caption).
3. Σβήνει τυχόν ζεύγη των οποίων το αρχείο εικόνας λείπει από τον φάκελο.
4. Αποθηκεύει στην προσωρινή μνήμη κάθε ζεύγος (ο μετασχηματισμένος tensor της εικόνας, περιγραφή) σε ένα αρχείο την πρώτη φορά που φορτώνεται (κάνω cache), έτσι ώστε οι επόμενες εποχές να παραλείπουν το βήμα αυτό της αργής αποκωδικοποίησης/αλλαγής μεγέθους PIL.

Μετά τον καθαρισμό, το χρησιμοποιήσιμο σύνολο δεδομένων περιείχε 6.070 ζεύγη εικόνας-περιγραφής.

2.2 Προεπεξεργασία εικόνων

Όλες οι εικόνες προ-επεξεργάστηκαν με την ακόλουθη διαδικασία της torchvision, που ταιριάζει με την τυπική μέθοδο εκπαιδευτικού DCGAN:

```
transforms.Compose([
    transforms.Resize(64),
    transforms.CenterCrop(64),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
])
```

Το βήμα Κανονικοποίησης μετατοπίζει τις τιμές των pixel από [0, 1] σε [-1, 1], κάτι που ταιριάζει με το εύρος εξόδου Tanh της Γεννήτριας και είναι τυπικό για την εκπαίδευση GAN.

2.3 Ρυθμίσεις του Dataloader

Το μέγεθος του batch ήταν 128. Οι workers ορίστηκαν σε (cpu_count - 2) έτσι ώστε η φόρτωση δεδομένων να είναι πλήρως παράλληλη (γρήγορη αλλά να μην επιβαρύνει πολύ το σύστημα). Το shuffling ήταν ενεργοποιημένο.

3. Κατασκευή Tokenizer

3.1 Γιατί έφτιαξα τον δικό μου tokenizer

Οι σύγχρονοι tokenizers LLM (BERT, GPT, T5) διαθέτουν λεξιλόγιο 30.000-50.000 tokens και είναι εκπαιδευμένοι σε ευρύ κείμενο του διαδικτύου. Για αυτό το πρόβλημα όπου υπάρχει ένας μικρός αριθμός περιγραφών των λουλουδιών, αυτό είναι άσκοπα μεγάλο και προσθέτει εκατομμύρια αχρησιμοποίητες παραμέτρους. Ως εκ τούτου, εκπάιδευσά έναν μικρό tokenizer WordPiece ειδικά στο στις συγκεκριμένες περιγραφές των λουλουδιών..

3.2 Λεπτομέρειες υλοποίησης

Έγινε χρήση της βιβλιοθήκης tokenizers της HuggingFace με ένα μοντέλο που ονομάζεται WordPiece:

- Κανονικοποίηση: εφαρμογή Unicode Normalization Form D (NFD), μετατροπή όλων των χαρακτήρων σε πεζά και αφαίρεση των τόνων (StripAccents)..
- Pre-tokenizer: έγινε διαχωρισμός με βάση τα κενά (διαχωρίστηκαν επίσης και τα σημεία στίξης). Επέλεξα κενά αντί για BertPreTokenizer επειδή οι λεζάντες λουλουδιών είναι απλές και δεν χρειάζονται τους αυστηρούς κανόνες του BERT.
- Ειδικά tokens: [UNK], [PAD], [CLS], [SEP], [MASK]. Παρόλο που μόνο τα [PAD] και [UNK] χρειάστηκαν για την υλοποίηση αυτού του έργου, ήταν καλύτερη η διατήρηση του πλήρους προτύπου του BERT που καθιστά τον tokenizer πιο εύκολα επεκτάσιμο και επαναχρησιμοποιήσιμο.
- Μέγεθος λεξιλογίου: Εκπαιδεύτηκε με στόχο τα 8.000 tokens. Το τελικό μέγεθος λεξιλογίου ήταν 7.288 tokens (το dataset περιείχε 6.947 μοναδικές λέξεις). Είναι απαραίτητος ο καθορισμός του συνόλου των tokens να είναι ελαφρώς πάνω από τον αριθμό μοναδικών λέξεων ώστε να δώσει στο WordPiece την ελευθερία να παράγει χρήσιμες συγχωνεύσεις υπολέξεων.

3.3 Επαλήθευση αποτελεσματικότητας του tokenizer

Μετά την εκπαίδευση, επαλήθευσα τον tokenizer με την πρόταση "this flower is orange and yellow in color with petals that are oval shaped. Photosynthesis". (η photosynthesis μπήκε επίτηδες και δεν υπάρχει γενικά σαν λέξη στο dataset) Η έξοδος ήταν:

```
['this', 'flower', 'is', 'orange', 'and', 'yellow',  
'in', 'color', ',', 'with', 'petals', 'that',  
'are', 'oval', 'shaped', ',',  
'photos', '##yn', '##thes', '##is']
```

Οι λέξεις που υπήρχαν στο λεξιλόγιο έμειναν ολόκληρες, ενώ η λέξη «photosynthesis» που ήταν εκτός λεξιλογίου χωρίστηκε σε τέσσερα τμήματα λέξεων με το πρόθεμα συνέχειας του WordPiece ##. Αυτό επιβεβαίωσε ότι ο tokenizer χειρίζεται σωστά το λεξιλόγιο.

4. Transformer Encoder (από την αρχή)

4.1 Στόχος

Ο κωδικοποιητής Transformer αντιστοιχίζει μια περιγραφή εικόνας σε ένα διάνυσμα 256 διαστάσεων σταθερού μεγέθους. Αυτό το διάνυσμα χρησιμοποιείται ως σήμα εισόδου (conditioning signal) για το GAN.

Υλοποίησα τον κωδικοποιητή από την αρχή ακολουθώντας το έργο του Vaswani et al. 2017, "Attention Is All You Need", ενότητες 3.4 και 3.5. Η πλήρης υλοποίηση βρίσκεται στο [transformer.ipynb](https://github.com/ashb25/transformer.ipynb).

4.2 Αρχιτεκτονική

Ο κωδικοποιητής αποτελείται από τέσσερα επίπεδα:

4.2.1 Token + Positional Embedding

Ένα επίπεδο `nn.Embedding` κάνει `map` τα token IDs των 256 διαστάσεων διανυσμάτων. Είναι σημαντικό ότι το `padding_idx` έχει οριστεί στο token ID [PAD], έτσι ώστε η τα `gradients` στην θέση του `pad` να είναι μηδέν.

Επιπλέον, προσθέτω την ημιτονοειδή κωδικοποίηση της θέσης (position encoding) από την αρχική εργασία, η οποία υπολογίζεται ως εξής:

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$$

Η κωδικοποίηση της θέσης υπολογίζεται μία φορά και αποθηκεύεται ως μη εκπαιδευσιμο buffer με `register_buffer`, επομένως πηγαίνει μαζί με το μοντέλο όταν μετακινείται στην GPU αλλά δεν βελτιστοποιείται.

4.2.2 Multi-Head Attention

Με `d_model=256` και 8 κεφαλές, κάθε κεφαλή προσοχής λειτουργεί σε 32 διαστάσεις (`d_k = 256/8 = 32`). Ο μηχανισμός attention βασίζεται στο κλιμακωτό εσωτερικό γινόμενο (scaled dot-product attention) και δίνεται από τον τύπο::

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Για τον υπολογισμό του μηχανισμού attention χρησιμοποιήθηκε η συνάρτηση `torch.nn.functional.scaled_dot_product_attention` του PyTorch, η οποία παρέχει βελτιστοποιημένη υλοποίηση του scaled dot-product attention και υποστηρίζει επιτάχυνση μέσω FlashAttention. Οι γραμμικοί μετασχηματισμοί για τα ερωτήματα (Q), κλειδιά (K), τιμές (V) και την τελική έξοδο (O) υλοποιήθηκαν χειροκίνητα.

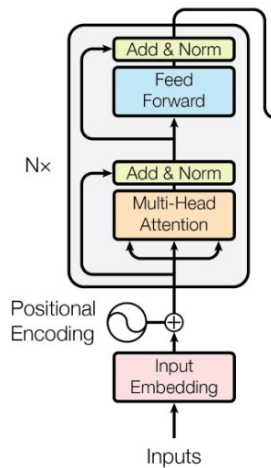
4.2.3 Position-wise Feed-Forward Network

Κάθε μπλοκ FFN είναι ένα MLP δύο επιπέδων που επεκτείνει το `d_model` à `4·d_model = 1024` διαστάσεις και πίσω. Ακολουθώντας τη σύγχρονη πρακτική (και μια ρητή σύσταση στο άρθρο GELU, Hendrycks & Gimpel 2016), χρησιμοποίησα το GELU αντί για το αρχικό ReLU επειδή το GELU παράγει μη μηδενικές κλίσεις για ελαφρώς αρνητικές εισόδους και αποφεύγει το πρόβλημα dying-ReLU που είναι συνηθισμένο στους κωδικοποιητές NLP..

4.2.4 Encoder Block

Το μπλοκ ακολουθεί την κλασσική δομή ενός transformer: Attention à Add&Norm à Feed-Forward Network (FFN) à Add&Norm. (Figure 1)

Και τα δύο επίπεδα Layer Normalization εφαρμόζονται μετά από κάθε post-norm όπως και στην αρχική εργασία των Transformers. Συνολικά χρησιμοποιήθηκαν τέσσερα blocks στοιβαγμένα. Η τελική αναπαράσταση των tokens υπολογίζεται με μέσο όρο κατά μήκος της διάστασης της ακολουθίας (mean pooling), ώστε να παραχθεί ένα ενιαίο διάνυσμα 256 διαστάσεων για κάθε περιγραφή εικόνας.



Εικόνα 1: Δομή encoder

4.3 Επαλήθευση

Πριν από την ενσωμάτωση με το GAN, δοκίμασα τον κωδικοποιητή σε ένα batch:

```
tokens shape : torch.Size([128, 75])
text_vector shape : torch.Size([128, 256])
```

Output:

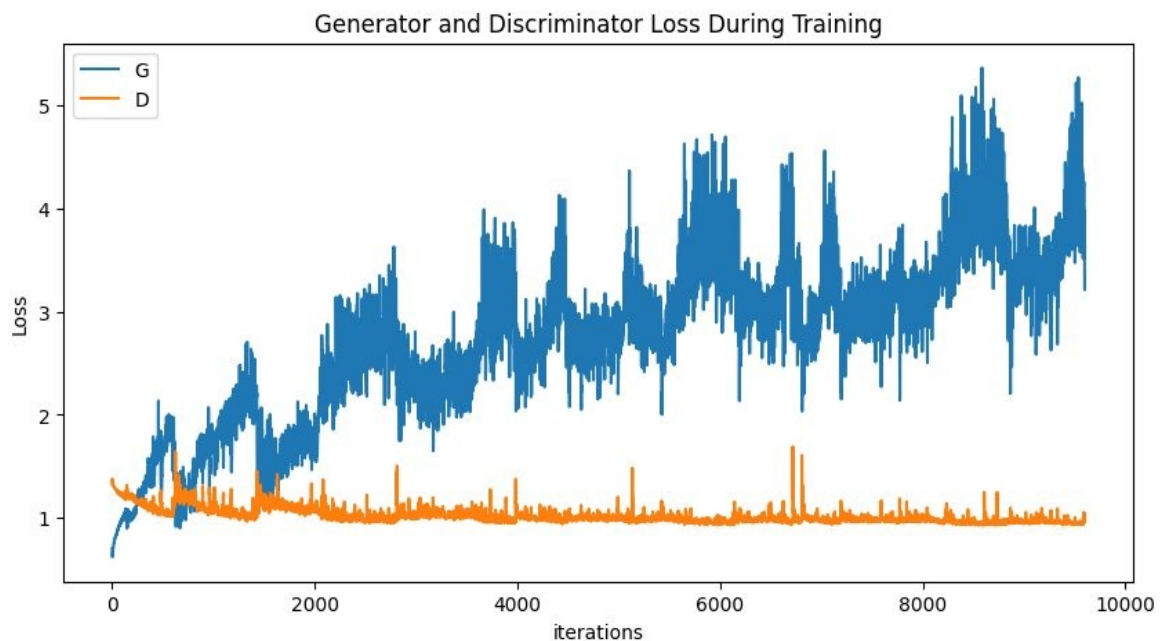
```
any nan : False
any inf : False
```

Πριν ξεκινήσει οποιαδήποτε εκπαίδευση του GAN, έγινε έλεγχος ώστε να επιβεβαιωθεί ότι το σχήμα της εξόδου ήταν σωστό, ότι δεν υπήρχαν μη έγκυρες τιμές όπως NaN ή Inf, και ότι η ροή της κλίσης λειτουργούσε κανονικά.

5. Conditional GAN (Φάση 1)

5.1 Vanilla DCGAN (μόνο demo)

Πριν ξεκινήσει η υλοποίηση του τελικού προβλήματος με τις συνθήκες κειμένου, έγινε χρήση μιας πρακτικής για ένα απλό DCGAN στο Flowers102 χρησιμοποιώντας τις ετικέτες του torchvision (cGan.ipynb). Αυτό χρησίμευσε ως ένας πρώτος έλεγχος για την αρχικοποίηση βαρών Generator, την απώλεια BCE και τη διαμόρφωση του βελτιστοποιητή Adam. Το test παρήγαγε σχετικά καλά αποτελέσματα εικόνων, το οποίο δείχνει ότι μπορεί να γίνει λυθεί το τελικό πρόβλημα.



Εικόνα 2: Έξοδος από την αρχική άνευ όρων επίδειξη DCGAN. Το μοντέλο παράγει εικόνες που μοιάζουν με λουλούδια ακόμη και χωρίς προσαρμογή κειμένου.

Μόλις το άνευ όρων DCGAN παρήγαγε αναγνωρίσιμα λουλούδια, προχώρησα στην υπό όρους έκδοση που ενσωματώνει τον κωδικοποιητή Transformer.

5.2 Αρχιτεκτονική generator

Είσοδος: ένα 100-διαστάσεων διάνυσμα θορύβου Gauss z και ένα 256-διαστάσεων διάνυσμα κειμένου που παρήγαγε ο encoder. Και τα δύο προβάλλονται σε ένα common latent space και χρησιμοποιούνται ως αρχική αναπαράσταση για τη δημιουργία της εικόνας:

Στη συνέχεια, ο generator εφαρμόζει διαδοχικά στρώματα ConvTranspose2d, τα οποία αυξάνουν σταδιακά τη χωρική ανάλυση της εικόνας:

- από 1×1 σε 4×4 ,
- μετά σε 8×8 ,
- 16×16 ,
- 32×32 ,

και τελικά σε εικόνα 64×64 pixels με 3 κανάλια χρώματος (RGB).

Μετά από κάθε στρώση υπάρχει ένα BatchNorm2d ακολουθούμενο από ReLU (εκτός από την τελευταία). Το τελευταίο layer της ενεργοποίησης είναι η Tanh, αντιστοιχίζοντας τις εξόδους σε $[-1, 1]$ για να ταιριάζουν με το κανονικοποιημένο εύρος της πραγματικής εικόνας.

5.3 Αρχιτεκτονική Discriminator

Ο Discriminator είναι αντίθετος από τον Generator (Conv2d αντί για ConvTranspose2d, LeakyReLU(0.2) αντί για ReLU, χωρίς BatchNorm στο πρώτο επίπεδο). Κρίσιμο είναι ότι το διάνυσμα του κειμένου εγχέεται στο επίπεδο 4*4, όπου η εικόνα έχει συμπιεστεί αρκετά ώστε να μπορούν να συγκριθούν αποτελεσματικά τα χαρακτηριστικά της με την πληροφορία του κειμένου.

Ο Discriminator λαμβάνει ένα από τα τρία ζεύγη (εικόνα, κείμενο) σε κάθε βήμα:

1. (real_image, correct_caption_vec) εκπαιδεύεται με σκοπό να βγάλει ~ 1 (real).
2. (real_image, wrong_caption_vec) εκπαιδεύεται με σκοπό να βγάλει ~ 0 (fake).
3. (generated_image, correct_caption_vec) εκπαιδεύεται με σκοπό να βγάλει ~ 0 (fake).

Το ζεύγος λανθασμένου κειμένου (περίπτωση 2) είναι απαραίτητο: χωρίς αυτό, ο Discriminator μπορεί να επιτύχει απλώς ελέγχοντας αν η εικόνα είναι πραγματική χωρίς να λαμβάνει υπόψη το διάνυσμα του κειμένου. Η συμπερίληψή του, αναγκάζει τον D να μάθει την κοινή κατανομή του συνδυασμού εικόνας και κείμενο.

5.4 Συνάρτηση σφάλματος

Σφάλμα Discriminator (με label smoothing real_label = 0.9):

```
L_D = BCE(D(x_real, t_correct), 0.9)
      + 0.5 * BCE(D(x_real, t_wrong), 0.0)
      + 0.5 * BCE(D(x_fake, t_correct), 0.0)
```

Το σφάλμα του Generator συνδιάζει δύο κανόνες:

```
L_G = BCE(D(x_fake, t_correct), 1.0) + λ_c * L_contrast
```

όπου το L_{contrast} είναι μια απώλεια αντίθεσης τύπου InfoNCE που εφαρμόζεται στα διανύσματα κειμένου ενός batch με στόχο παρόμοια δείγματα να έχουν κοντινές αναπαραστάσεις και διαφορετικά δείγματα πιο απομακρυσμένες:

```
def contrastive_loss(vecs, temperature=0.07):
    vecs_norm = F.normalize(vecs, dim=1)
    sim = vecs_norm @ vecs_norm.T / temperature
    labels = torch.arange(vecs.size(0)).to(vecs.device)
    return F.cross_entropy(sim, labels)
```

Αυτός ο όρος δίνει ποινή στον κωδικοποιητή για την παραγωγή του ίδιου διανύσματος για διαφορετικές περιγραφές. Χωρίς αυτόν, ο κωδικοποιητής κατέρρεε επανειλημμένα σε μια σταθερή έξοδο κατά τη διάρκεια των πρώτων πειραμάτων (βλ. ενότητα 6).

5.5 Επιπλέον λεπτομέρειες εκπαίδευσης

- Adam optimisers με $\text{lr} = 2 \cdot 10^{-4}$, $\text{beta1} = 0.5$, $\text{beta2} = 0.999$ (standard DCGAN ρύθμιση).
- Τα βάρη προσαρμόστηκαν με custom συνάρτηση: Conv weights $\sim N(0, 0.02)$, BatchNorm weights $\sim N(1, 0.02)$. Έκανα αλλαγές στις PyTorch's την προεπιλεγμένη Kaiming-uniform τεχνική διότι εμπειρικά παρατηρήθηκε ότι το DCGAN training ήταν πολύ ασταθές.
- Προστίθεται θόρυβος στις εισόδους του D και παρατηρήθηκε μείωση από 0,05 σε 0 κατά τις πρώτες 30.000 επαναλήψεις. Αυτό εμποδίζει το D να επιτύχει ακρίβεια 100% πολύ νωρίς
- Gradient clipping με $\text{max_norm} = 1.0$ σε όλα τα τρία networks.
- Με στόχο να διατηρηθεί ο encoder αποδοτικός έγινε παράλειψη του encoder optimizer βήματος εάν η τυπική απόκλιση διανύσματος κειμένου-παρτίδας πέσει κάτω από 0,1.
- Γινόταν αποθήκευση των αποτελεσμάτων κάθε 500 iterations.

6. Εκπαίδευση Σταδίου πρώτου: Διαδικασία, Αποτυχίες και Τελικά Αποτελέσματα

6.1 Τι λειτούργησε την πρώτη φορά

Το tokenization, ο encoder forward pass, και το dataloader pipeline μπήκαν κανονικά στο σύνολο διότι η κάθε διαδικασία είχε γραφτεί σε ξεχωριστά Jupyter python notebooks.

6.2 Σφάλματα που παρατηρήθηκαν κατά την εκπαίδευση

Η πρώτη εκπαίδευση δεν λειτούργησε την πρώτη φορά. Υπήρξαν πέντε κύριες επαναλήψεις εντοπισμού σφαλμάτων πριν φτάσουμε στο τελικό μοντέλο της Φάσης 1. Τις καταγράφω όλες εδώ επειδή οι τρόποι αστοχίας/λαθών αποτελούν σημαντικό μέρος αποφάσεων και αλλαγών που ακολούθησα που μπορεί σε μία πρώτη ανάγνωση να φαίνεται ασαφής ο λόγος.

6.2.1 Πρώτη προσπάθεια: Κυριαρχία του D, κατάρρευση του G

Με την αρχική αρχιτεκτονική, η απώλεια BCE του Discriminator πήγε στο ~ 0 εντός 100 επαναλήψεων και η απώλεια του Generator απέκλινε. Το D κέρδιζε εξ' ολοκλήρου τον G κάτι που δεν επέφερε καλά αποτελέσματα πίσω στο G. Το πρόβλημα αυτό αντιμετωπίστηκε με εξομάλυνση ετικέτας (real_label 1.0 $\rightarrow$ 0.9), θόρυβο και μικραίνοντας τον ρυθμό μάθησης του discriminator.

6.2.2 Δεύτερη προσπάθεια: περίπλοκη αρχιτεκτονική

Δοκιμάστηκε επίσης μια πιο σύνθετη προσέγγιση, όπου το διάνυσμα κειμένου εισαγόταν σε κάθε στρώση του G (text_proj1...text_proj5), ο G εκπαιδευόταν δύο φορές για κάθε βήμα του D και προστέθηκαν οι απώλειες LSGAN και diversity loss. Αυτό οδήγησε σε μια σύντομη σημαντική ανακάλυψη στο iter 43.500 όπου εμφανίστηκαν σχήματα λουλουδιών, αλλά το σύστημα στη συνέχεια δεν ακολούθησε ορθή πορεία και η ποιότητα της εικόνας υποχώρησε. Η υπερβολικά επεξεργασμένη αρχιτεκτονική ήταν δύσκολο να αιτιολογηθεί και ήταν ασταθής.

6.2.3 Τρίτη προσπάθεια: ένα σφάλμα που ήταν “κρυμμένο” στα data

Η λογική fallback της cache στην κλάση του dataset έκρυβε το γεγονός ότι οι αρχικές εικόνες .jpg είχαν διαγραφεί μεταξύ των επαναλήψεων. Αφού επιβεβαίωσα ότι η cache ήταν σε καλή κατάσταση (6.070 δείγματα, [3, 64, 64], εύρος [-1, 1]), διαπίστωσα ότι το dataset δεν ήταν το πρόβλημα.

6.2.4 Τέταρτη προσπάθεια: encoder collapse

Σε μια απλοποιημένη ανακατασκευή που επέστρεψε σε προετοιμασία με μία μόνο ένεση και απώλεια BCE με το αρνητικό λάθος κειμένου, παρατήρησα δύο τρόπους αστοχίας κατά τη διάρκεια των πρώτων 500 επαναλήψεων:

- Ο κωδικοποιητής κατέρρευσε (η ομοιότητα συνημίτονου μεταξύ διαφορετικών προτροπών αυξήθηκε στο 0,99+), επομένως όλες οι εικόνες που παράγαγε ο Generator έμοιαζαν ίδιες.
- Μόλις κατέρρευσε, ο encoder δεν ανέκαμψε, παρόλο που η απώλεια αντίθεσης θα έπρεπε να τον είχε τιμωρήσει.

Βασική αιτία: Η αντίθεση $\lambda = 0,1$ ήταν πολύ ασθενής σε σχέση με την κλίση BCE-έναντι-D που τράβηξε τον κωδικοποιητή προς όποιο σταθερό διάνυσμα βοήθησε περισσότερο το D. Το σήμα αντίθεσης ήταν περίπου 0,3% του συνολικού μεγέθους κλίσης.

6.2.5 Πέμπτη προσπάθεια: το warmup που ήταν τελικά επιβλαβές

Για να διορθώσω την κατάρρευση του κωδικοποιητή, προσπάθησα να προεκπαιδεύσω τον κωδικοποιητή για 500 βήματα χρησιμοποιώντας μόνο contrasting losses, πριν ξεκινήσω την εκπαίδευση GAN. Αυτό ήταν λάθος: οι αντιθετικές απώλειες έχουν έναν ασήμαντο minimizer (η επιστροφή πανομοιότυπων διανυσμάτων για όλες τις εισόδους θα έδινε cross entropy = $\log(B)$), αλλά η ελεύθερη βελτιστοποίηση σε ένα μικρό επαναλαμβανόμενο σύνολο επέτρεπε στον κωδικοποιητή να

απομνημονεύει εξόδους ανά θέση που ελαχιστοποιούν την απώλεια σε $\approx 0,0004$). Όταν η πραγματική εκπαίδευση GAN ξεκίνησε με ανακατεμένα batches, η λύση του κωδικοποιητή απέτυχε και κατέρρευσε αμέσως.

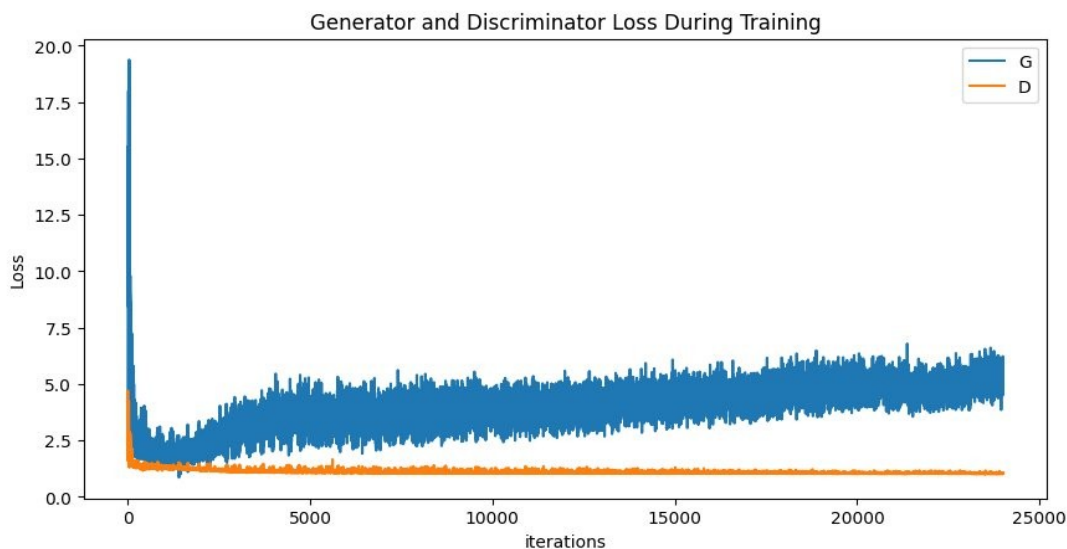
6.2.6 Τελική διόρθωση

Η επιτυχής τελική διαμόρφωση του συστήματος έγινε με:

1. Αφαίρεση της προθέρμανσης. Εκπαίδευση και των τριών δικτύων από κοινού από τυχαία αρχική τιμή.
2. Αύξηση του $\lambda_{\text{contrast}}$ από 0,1 σε 0,5, ώστε η loss of contrast να παρέχει ουσιαστική πίεση (~12% της συνολικής κλίσης αντί για ~3%).
3. Εισαγωγή ενός μόνο διανυσματικού κειμένου (G: στην είσοδο· D: στη στρώση 4*4).
4. Χρήση της απώλειας BCE παντού (το LSGAN δεν χρησιμοποιήθηκε τελικά).
5. Αποθήκευση κάθε 500 επαναλήψεις για μεγαλύτερη επεξηγησιμότητα του μοντέλου και των λάθους για βελτιώσεις

6.3 Τελική εκπαίδευση

Με τη διορθωμένη διαμόρφωση, η εκπαίδευση ήταν σταθερή για όλες τις 24.000 επαναλήψεις (500 εποχές).



Εικόνα 3: Καμπύλη σφάλματος από την τελική εκπαίδευση. Το loss του Discriminator είναι σταθερό γύρω στο 1,0. Το loss του Generator αυξάνεται σταθερά υποδεικνύοντας ότι υπήρχε πίεση και το μοντέλο μάθαινε

Ο κωδικοποιητής παρέμεινε σε καλή κατάσταση καθ' όλη τη διάρκεια: η ομοιότητα συνημιτόνου μεταξύ διαφορετικών κειμένων (prompts) παρέμεινε κάτω από 0,6 στο 95% στα checkpoints, με το text_std σταθερά πάνω από 0,4. Το κατώφλι που εισήχθη ώστε ο κωδικοποιητής να μην καταστραφεί λειτούργησε και βοήθησε σε σταθερά αποτελέσματα

6.4 Ποιότητα αποτελεσμάτων πρώτης φάσης

Η μέγιστη ποιότητα Φάσης 1 επιτεύχθηκε περίπου στην 8.500η επανάληψη (εποχή 177). Παρακάτω είναι τα δείγματα των αποτελεσμάτων εκπαίδευσής των καλύτερων iterations:



Εικόνα 4(iter 10000): Το μοντέλο έμαθε το κόκκινο χρώμα, δεν παράγει πολύ καλές εικόνες το Prompt δεν ακολουθείτε πλήρως



Εικόνα 5: τα λουλούδια φαίνονται καλύτερα αλλά ακόμα το prompt δεν ακολουθείτε



Το μοντέλο φαίνεται να μπορεί να ξεχωρίσει το διαφορετικό κείμενο αλλά σε μικρά κείμενα όπως red και purple φαίνεται να μην μπορεί να δώσει έμφαση στην διαφορά τους. Μετά την επανάληψη ~14.000, η εκπαίδευση άρχισε να παράγει εικόνες με ένα περίεργο μοτίβο (ορατά στην επανάληψη 22.000), κάτι που οφείλεται στο ConyTranspose2d. Το σημείο ελέγχου της επανάληψης 8.500 επιλέχθηκε επομένως ως το τελικό σημείο της Φάσης 1.

Εικόνα 6 (iter 8500): Βελτίωση στην ποιότητα των λουλουδιών και μεγαλύτερη προσοχή του μοντέλου στο prompt



Εικόνα 7 (iter 22000): στην επανάληψη 22,000 το "red layered" prompt παράγαγε ένα κίτρινο λουλούδι με ένα μοτίβο που θυμίζει σκακιέρα (αριστερά), ενώ οι άλλες εικόνες παραμένουν λογικές. Αυτός είναι ο λόγος για τον οποίο η επανάληψη 8.500 επιλέχθηκε ως το τελικό σημείο ελέγχου Φάσης 1

6.5 Περιορισμοί που εμφανίστηκαν στην πρώτη φάση

Δύο συγκεκριμένες αδυναμίες του μοντέλου Φάσης 1 εντοπίστηκαν μέσω ποσοτικών μετρήσεων και παρατηρώντας οπτικά:

- Αδύναμη ευθυγράμμιση κειμένου-εικόνας ως προς το χρώμα. Τα "κόκκινα" μηνύματα συχνά παράγουν κίτρινα ή πορτοκαλί λουλούδια. Το "μπλε" σχεδόν πάντα παράγει λευκό ή ροζ. Ο κωδικοποιητής μαθαίνει ξεχωριστά διανύσματα ανά λεζάντα, αλλά η γεννήτρια δεν τα μεταφράζει πάντα στο χρώμα που ζητείται.
- Σύμπτυξη λειτουργίας στον άξονα θορύβου. Διαφορετικά διανύσματα θορύβου για το ίδιο μήνυμα παράγουν σχεδόν πανομοιότυπες εικόνες (μέσος όρος MAE ανά ζεύγη pixel = 0,0114). Η γεννήτρια ουσιαστικά αντιμετωπίζει το z ως μια μικρή διαταραχή και όχι ως πραγματική πηγή διακύμανσης.

Αυτές οι αδυναμίες καθόρισαν τον στόχο της Φάσης 2: βελτίωση του μοντέλου στην έμφαση στο κείμενο και στην εικόνα μαζί μέσω της Ενισχυτικής Μάθησης.

7. Εκπαίδευση Φάσης 2: Ενισχυτική Μάθηση (Reinforcement Learning)

7.1 Γιατί χρειαζόμαστε RL σε αυτήν την εργασία

Η Φάση 1 άφησε μια σαφή αδυναμία: οι έξοδοι της γεννήτριας αντιστοιχούσαν μόνο εν μέρει στο κείμενο που ζητήθηκε, ειδικά όσον αφορά το χρώμα. Δύο επιλογές ήταν διαθέσιμες:

1. Επανεκπαίδευση της Φάσης 1 με μια ισχυρότερη αρχιτεκτονική επεξεργασίας κειμένου (π.χ. να προσθέσουμε attention σε κάθε layer του G).
2. Βελτιστοποίηση του υπάρχοντος μοντέλου με RL.

Για την ικανοποίηση των απαιτήσεων της εργασίας έγινε επιλογή της δεύτερης «λύσης»: για την δεύτερη λύση έγινε χρήση ενός προ-εκπαιδευμένου μοντέλου οπτικής γλώσσας (CLIP) για να παρέχει μια άμεση ανταμοιβή. Είναι επίσης το σύγχρονο παράδειγμα που χρησιμοποιείται για την ευθυγράμμιση(alignment) μεγάλων γενετικών μοντέλων (RLHF στο InstructGPT, DDPO για μοντέλα διάχυσης, RealGen για μετατροπή κειμένου σε εικόνα).

7.2 Διατύπωση του προβλήματος

Το fine-tuning του generator το έχουμε διατυπώσει σαν πρόβλημα εκπαίδευσης πολιτικής (reinforcement learning style):

- **Policy:** ο Generator $G_\theta(z, c)$, μια ντετερμινιστική συνάρτηση από το latent state (z , encoded text c) στο action (image x). (δηλαδή ο Generator μας παίρνει έναν τυχαίο θόρυβο και το κωδικοποιημένο κείμενο, και παράγει μια εικόνα)
- **Reward:** $R(x, \text{text}) = \cos(\phi_{\text{img}}(x), \phi_{\text{txt}}(\text{text}))$, Το CLIP υπολογίζει το cosine similarity του πόσο ταιριάζουν οι εικόνες με την περιγραφή.
- **Objective:** $J(\theta) = E[R(G_\theta(z, c(\text{text})), \text{text})]$, μεγιστοποιείται από την gradient ascent.

Επειδή και το policy και το reward είναι διαφορίσιμες, το Deterministic Policy Gradient (Silver et al. 2014) μπορεί να υπολογιστεί απευθείας με backpropagation:

$$\nabla_\theta J = E[\nabla_x R(x, \text{text}) \cdot \nabla_\theta G_\theta(z, c(\text{text}))]$$

Για την εργασία υλοποίησα επίσης μια παραλλαγή με στοχαστική πολιτική (με προσθήκη Gaussian θορύβου στην έξοδο του generator) για σύγκριση η ντετερμινιστική έκδοση είχε χαμηλότερη διακύμανση και παρήγαγε πιο ομαλές καμπύλες εκπαίδευσης, οπότε έγινε η κύρια μέθοδος.

7.3 Πειραματική Υλοποίηση

RL fine-tuning loop:

1. Φορτώνουμε τα βάρη από την καλύτερη επανάληψη (iter 8500 checkpoint) για τον Generator, Discriminator και Encoder.
2. Παγώνουμε τον Encoder, τον Discriminator, και το CLIP. Στόχος είναι μόνο ο generator να μπορεί να εκπαιδευτεί.
3. Αλλάζουμε τον Adam optimiser για τον Generator με $lr = 2 \cdot 10^{-5}$ (10* ποιο χαμηλό από αυτός της φάσης 1). Δεν μπορούσε να γίνει χρήση του optimizer της φάσης 1 διότι η μνήμη που είχε από τα προηγούμενα gradients του έδωσε ένα momentum το οποίο δεν θέλουμε για την φάση 2.
4. Ανά batch κωδικοποιούμε την λεζάντα με τον tokeniser του CLIP, αλλάζουμε το μέγεθος της παραγόμενης εικόνας από 64*64 σε 224*224 (το αναμενόμενο μέγεθος εισόδου του CLIP)· εφαρμόζουμε την τυπική κανονικοποίηση του CLIP, υπολογίζουμε τα χαρακτηριστικά εικόνας και κειμένου, η cosine ομοιότητα αποτελεί την ανταμοιβή. Τα χαρακτηριστικά της

εικόνας διαπερνούν το CLIP και φτάνουν μέχρι τον Generator, τα βάρη του CLIP παραμένουν παγωμένα (δεν εκπαιδεύονται).

5. Συνολική συνάρτηση απώλειας του Generator = $\lambda_{RL} * clip_loss + 0.1 * realism_loss$, όπου $realism_loss = BCE(D(x_{fake}, text), 1)$, χρησιμοποιώντας τον παγωμένο Discriminator με σκοπό να εμποδίζει τον Generator να παράγει εικόνες που ικανοποιούν το CLIP αλλά δεν μοιάζουν με λουλούδια..
6. $\lambda_{RL} = 10$ (απαιτείται μεγάλη βαρύτητα διότι ο όρος realism με BCE έχει πολύ μεγαλύτερη αριθμητική κλίμακα σε σχέση με τον όρο CLIP που βασίζεται σε cosine ομοιότητα).
7. Clipping των gradients με $max_norm = 0.5$ (πιο σφιχτό από τη Φάση 1).

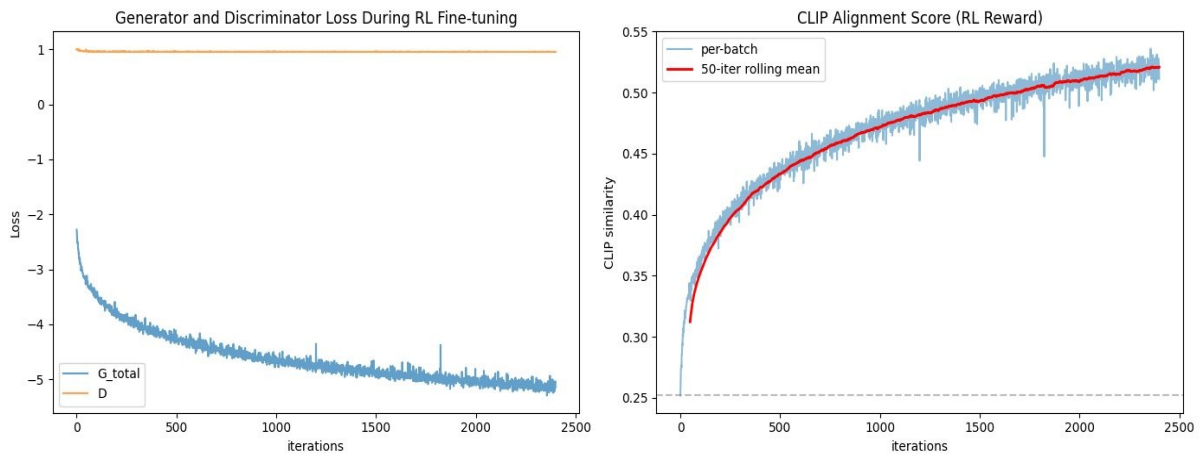
7.4 Γιατί πάγωσα τον Discriminator

Ξεκινώντας την ανάλυση από την πρώτη προσπάθεια έκανα RL με τον Discriminator να μην είναι παγωμένος. Δεν μπορούσε όμως να διατηρηθεί η ισορροπία και ο Discriminator κυριαρχούσε έναντι του generator. Ο Discriminator δεν είχε να μάθει τίποτα και είδη από την φάση 1 ξεκίνησε να συγκλίνει. Το score του Generator στις ψεύτικες εικόνες ήταν 0,0003. Το CLIP μπορούσε να συνεισφέρει 0,025 από 8,8 στο total loss οπότε ουσιαστικά δεν άλλαζε τίποτα.

Αυτό το πρόβλημα λύθηκε με το πάγωμα του D στην δεύτερη φάση. Το D μειώνεται πλέον σταθερά και το CLIP κυριαρχεί σαν πηγή για τα gradients.

8. Σύγκριση και αποτελέσματα της δεύτερης φάσης

8.1 Εκπαίδευση



Εικόνα 8: Καμπύλες εκπαίδευσης Φάσης 2. Αριστερά: συνολική απώλεια γεννήτριας (καθοδηγούμενη από CLIP, άρα αρνητική). Δεξιά: Η βαθμολογία ευθυγράμμισης CLIP ανεβαίνει μονοτονικά από την αρχική τιμή Φάσης 1 των 0,252 σε πάνω από 0,52, με κυλιόμενο μέσο όρο 50 επα

Η βαθμολογία ευθυγράμμισης CLIP αυξήθηκε από 0,252 (βάση Φάσης 1) σε 0,520 μετά από 2.400 επαναλήψεις RL: μια σχετική βελτίωση +106%. Η καμπύλη είναι ομαλή και μονότονη, με την πιο απότομη βελτίωση στις πρώτες 500 επαναλήψεις και ένα οροπέδιο που σχηματίζεται γύρω στις 1.800 επαναλήψεις. Δεν υπάρχει αιχμή στο reward-hacking (όπου η ευθυγράμμιση θα ανέβαινε ξαφνικά αλλά η ποιότητα της εικόνας θα κατέρρεε).

Ο “όρος ρεαλισμού” (παγωμένο-D BCE) μειώθηκε από 2,4 σε 0,7 κατά την εκπαίδευση. Αντίθετα με τη διαισθητική, αυτό σημαίνει ότι οι έξοδοι της Γεννήτριας έγιναν πιο ρεαλιστικές σύμφωνα με το D, όχι λιγότερο, ακόμη. Αυτό είναι ένα ιδανικό αποτέλεσμα στο οποίο συνήθως η βελτιστοποίηση σε μια μη ανταγωνιστική ανταμοιβή προκαλεί κάποια οπισθοδρόμηση ρεαλισμού.

8.2 Ποσοτική σύγκριση

Μετρική	Φάση 1	Φάση 2	Διαφορές
CLIP alignment	0.252	0.520	+106%
Noise diversity (MAE)	0.0114	0.0089	-22%
Text sensitivity (MAE)	0.378	0.352	-7%
Text/Noise ratio	33.1	39.5	+19%

Πίνακας 1: Ποσοτικές μετρήσεις Φάσης 1 έναντι Φάσης 2. Όσο υψηλότερο είναι το αποτέλεσμα, τόσο καλύτερο για την ευθυγράμμιση CLIP, την ποικιλομορφία θορύβου και τον λόγο T/N

Τι παρατηρείται με βάση τον πίνακα:

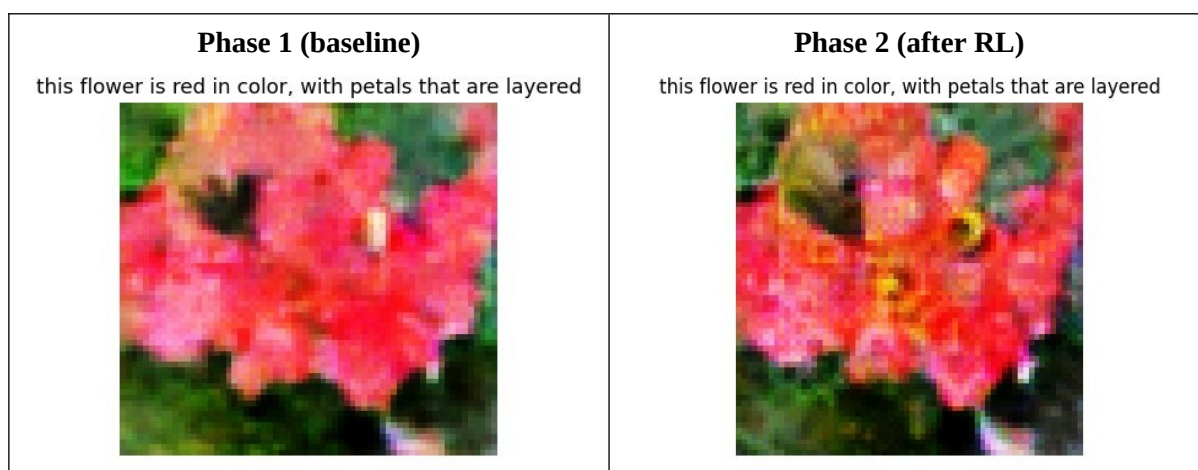
- Η ευθυγράμμιση CLIP υπερδιπλασιάστηκε. Αυτό είναι ένα μεγάλο θετικό στοιχείο.
- Η ποικιλομορφία θορύβου μειώθηκε κατά 22%. Και οι δύο φάσεις βρίσκονται σε καθεστώς κατάρρευσης ($MAE < 0,02$ σημαίνει ότι οι εικόνες είναι ουσιαστικά πανομοιότυπες για διαφορετικό θόρυβο). Η ενισχυτική μάθηση (RL) επιδείνωσε ελαφρώς το φαινόμενο, καθώς

η ανταμοιβή του CLIP λειτουργεί ως ισχυρός μηχανισμός έλξης προς μία «κανονική» εικόνα με τη βέλτιστη βαθμολογία CLIP για κάθε περιγραφή.

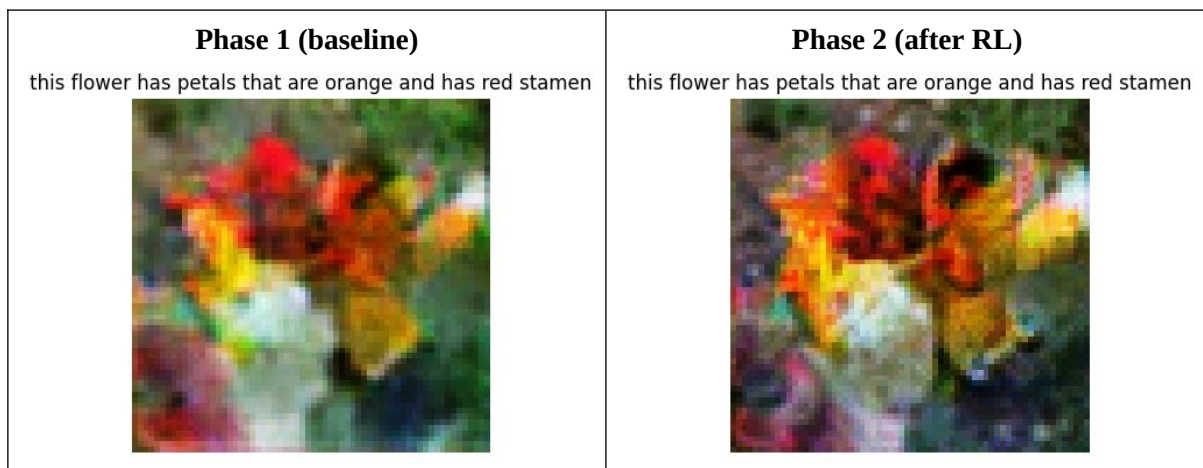
- Η ευαισθησία στο κείμενο μειώθηκε κατά 7%. Οι έξοδοι της Φάσης 2 τείνουν να συγκλίνουν σε ένα κοινό οπτικό ύφος το οποίο ευνοείται από το CLIP. Οι αλλαγές είναι τα κορεσμένα χρώματα και υφές φυσικών εικόνων με αποτέλεσμα η απόσταση pixel-MAE μεταξύ διαφορετικών προτροπών να είναι ελαφρώς μικρότερη, παρότι η σημασιολογική ευθυγράμμιση βελτιώνεται.
- Ο λόγος κειμένου/θορύβου βελτιώθηκε κατά 19%, που σημαίνει ότι το κείμενο συνεισφέρει περισσότερο από τον θόρυβο στην έξοδο ενός μοντέλου Φάσης 2 σε σχέση με ένα μοντέλο Φάσης 1.

8.3 Ποιοτική σύγκριση

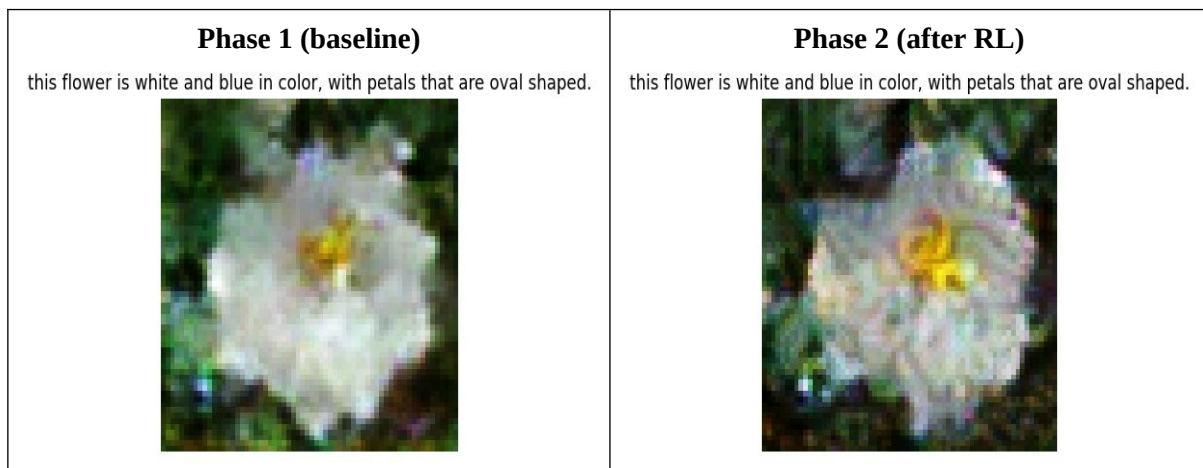
Έξοδοι Φάσης 1 έναντι Φάσης 2 για ίδιο θόρυβο, ίδιο κωδικοποιητή όμως διαφέρουν μόνο τα βάρη της γεννήτριας:



Εικόνα 9: "this flower is red in color, with petals that are layered". Στην φάση 1 το λουλούδι έχει ένα κοραλί χρώμα. Στην φάση 2 τα χρώματα είναι πιο έντονα και το το χρώμα είναι πιο κόκκινο ενώ φαίνεται στο κέντρο το κίτρινο του λουλουδιού και τα πέταλα



Εικόνα 10: "this flower has petals that are orange and has red stamen". Στην φάση 1 έχουμε ένα πορτοκαλοκόκκινο λουλούδι αρκετά θωλό όμως και δύσκολο να διακριθεί. Στην φάση 2 έχουμε διαυγές πορτοκαλί πέταλα με ορατά κόκκινη και κίτρινη δομή στημόνα στο κέντρο



Εικόνα 11: "this flower is white and blue in color, with petals that are oval shaped". Σε αυτό το παράδειγμα το μπλε της περιγραφής αγνοείται και στις δύο φάσεις. Η διαφορά μόνο είναι ότι στην δεύτερη φάση το άσπρο λουλούδι έχει μεγαλύτερη ευκρίνεια με πέταλα

8.4 Συγκριτικά αποτελέσματα των δύο φάσεων (οπτικά)

Στην φάση 2 οι εικόνες παρουσιάζονται με υψηλότερη ευκρίνεια σε σύγκριση με της φάσης 1. Ο σκοπός του CLIP ήταν να δώσει εικόνες σαν αυτή στο αποτέλεσμα καθώς έχει εκπαιδευτεί σε εκατοντάδες εκατομμύρια φυσικές εικόνες στο διαδίκτυο και επομένως προσπαθεί μέσω τις εκπαίδευσης να εισάγει αυτά τα στατιστικά στοιχεία φυσικής εικόνας, τα οποία σε ανάλυση 64*64 εκδηλώνονται ελαφρώς με κάποια μοτίβα. Σε υψηλότερες αναλύσεις αυτή θα ήταν ρεαλιστική όψη αλλά λόγω του 64*64 είναι οριακά θόρυβος. Αυτό συνάδει με την εργασία που μελέτησα που είχε καθοδήγηση από το CLIP ((Black et al. 2024 "DDPO," Lee et al. 2023 "Aligning Text-to-Image Models using Human Feedback").

10. Τελικά συμπεράσματα

Αυτό το έργο είχε ως στόχο να ενσωματώσει τρία παραδείγματα βαθιάς μάθησης: Συνελικτικά δίκτυα, transformers και ενισχυτική μάθηση σε ένα ενιαίο σύστημα που θα πάρει ένα κείμενο για λουλούδια και θα παράξει το αντίστοιχο λουλούδι που ζητήθηκε.

Οι τεχνικές οι αρχιτεκτονικές που υλοποιήθηκαν από την αρχή: ο tokenizer τύπου WordPiece, ο Transformer encoder με multi-head attention σύμφωνα με τους Ashish Vaswani et al. (2017), το conditional DCGAN με εκπαίδευση adversarial learning πάνω σε mismatched text pairs, η contrastive loss InfoNCE για την κανονικοποίηση του encoder και ο βρόχος fine-tuning ενισχυτικής μάθησης (RL) με deterministic policy gradients, χρησιμοποιώντας το CLIP ως frozen reward model. Το μόνο προεκπαιδευμένο στοιχείο είναι το CLIP, το οποίο χρησιμοποιείται αποκλειστικά ως εξωτερικός oracle ευθυγράμμισης στη Φάση 2.

Το υπό όρους GAN Φάσης 1 παράγει αναγνωρίσιμες, πολύχρωμες εικόνες λουλουδιών με μερική προσαρμογή κειμένου, μετά από ουσιαστική διόρθωση σφαλμάτων όπως την κυριαρχία του Discriminator, την κατάρρευση της αναπαράστασης του κωδικοποιητή και μια αντιπαραγωγική διαδικασία προθέρμανσης του Generator. Η βελτιστοποίηση RL με καθοδήγηση CLIP Φάσης 2 βελτιώνει την «ευθυγράμμιση» κειμένου-εικόνας κατά ένα μέτρο +106% (βαθμολογία CLIP 0,252 έως 0,520) σε 2.400 επαναλήψεις RL, με το ρεαλιστικό κομμάτι να διατηρείται με έναν παγωμένο Discriminator.

Ο συνολικός χρόνος που επενδύθηκε από την αρχική απλή επίδειξη DCGAN, μέσω των πολλαπλών επαναλήψεων εντοπισμού σφαλμάτων, έως τη φάση βελτιστοποίησης RL αντικατοπτρίζεται σε όλη τη συζήτηση της έκθεσης σχετικά με τους τρόπους αστοχίας. Η διαδικασία δεν είναι αντίγραφο κάποιας μεμονωμένης εργασίας, είναι μια σκόπιμη ενσωμάτωση τεχνικών από τους Reed et al. 2016 (GANs μετατροπής κειμένου σε εικόνα με εκπαίδευση σε λάθος κείμενο), Vaswani et al. 2017 (transformers), Goodfellow et al. 2014 και Radford et al. 2016 (DCGAN), Silver et al. 2014 (ντετερμινιστική διαβάθμιση πολιτικής), Radford et al. 2021 (CLIP), και η πιο πρόσφατη βιβλιογραφία RLHF / RL-for-T2I, εφαρμοσμένη σε ένα αναπαραγωγίμο πρόβλημα δημιουργίας λουλουδιών.

11. Βιβλιογραφία

- Goodfellow, I. J., et al. (2014). Generative Adversarial Nets. *Advances in Neural Information Processing Systems*.
- Radford, A., Metz, L., & Chintala, S. (2016). Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks (DCGAN). *ICLR*.
- Reed, S., Akata, Z., Yan, X., Logeswaran, L., Schiele, B., & Lee, H. (2016). Generative Adversarial Text to Image Synthesis. *ICML*.
- Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*.
- Hendrycks, D., & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). *arXiv:1606.08415*.
- Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic Policy Gradient Algorithms. *ICML*.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*.
- Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP). *ICML*.
- Yu, L., Zhang, W., Wang, J., & Yu, Y. (2017). SeqGAN: Sequence Generative Adversarial Nets with Policy Gradient. *AAAI*.
- Li, Y., Liu, Z., Zhao, X., et al. (2024). TextCraftor: Your Text Encoder Can be Image Quality Controller. *arXiv:2403.18978*.
- Mao, Q., Lee, H. Y., Tseng, H. Y., Ma, S., & Yang, M. H. (2019). Mode Seeking Generative Adversarial Networks for Diverse Image Synthesis. *CVPR*.
- Black, K., Janner, M., Du, Y., Kostrikov, I., & Levine, S. (2024). Training Diffusion Models with Reinforcement Learning (DDPO). *ICLR*.
- Lee, K., Liu, H., Ryu, M., et al. (2023). Aligning Text-to-Image Models using Human Feedback. *arXiv:2302.12192*.
- Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback (InstructGPT). *NeurIPS*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.
- HuggingFace LLM Course, Chapter 6: Tokenizers. <https://huggingface.co/learn/llm-course/chapter6/8>.
- PyTorch DCGAN Tutorial. https://docs.pytorch.org/tutorials/beginner/dcgan_faces_tutorial.html.